RADAR STOCK

Pauta de trabajo paso a paso para no perderse durante el desarrollo

Esta pauta acompaña al informe de decisiones de diseño. Úsala como lista de verificación: avanza en orden, marca cada casilla cuando algo funcione de verdad (probado, no solo escrito) y no pases a la siguiente parte hasta cumplir la sección «Listo cuando». Las fechas son una referencia basada en la Carta Gantt; verifícalas contra el archivo original.

1. Seis reglas de oro

1. **Una tarea por vez.** Cada prompt pide una sola cosa pequeña. Nunca «construye toda la app».
2. **Probar antes de seguir.** Si no lo viste funcionar en el emulador, el navegador o phpMyAdmin, no está terminado.
3. **Leer lo que cambió el agente.** Si no entiendes una parte, pídele que te la explique antes de aceptarla. Tú debes poder explicar tu proyecto frente al docente.
4. **Un commit por tarea terminada.** Así siempre tienes un punto seguro al que volver.
5. **Alcance cerrado.** Solo RF001-RF006. Si surge una idea nueva, anótala en Trello como «después», no la construyas.
6. **Obligatorio primero, extras después.** El escáner y el resumen de ventas se hacen al final de su parte, y son lo primero que se posterga si el tiempo se acorta.

2. Preparación del entorno (antes de la Parte 1)

- ☐ **Git y GitHub:** repositorio clonado en tu computador y *git status* funciona. Nombre y correo configurados en Git.
- ☐ **XAMPP:** instalado; Apache y MySQL aparecen en verde; *http:///localhost/phpmyadmin* abre en el navegador.
- ☐ **Android Studio:** instalado, con el SDK de Android y un emulador (AVD) creado que arranca sin errores.
- ☐ **Celular Android** con la depuración por USB activada (muy recomendado: el escáner de cámara se prueba mejor en un teléfono real).
- ☐ **Antigravity** con la carpeta del repositorio abierta (o Visual Studio Code si lo prefieres para PHP y la landing).
- ☐ **Trello:** un tablero con una tarjeta por cada Parte (1 a 6) y columnas Por hacer, En curso y Hecho.
- ☐ **Carpetas del repo:** database/, backend/, android/ y landing/ (database/ ya existe).

3. El ciclo de cada tarea (repítelo siempre)

1. **Elegir** la siguiente tarea de la parte en curso y moverla a «En curso» en Trello.
2. **Crear una rama** para esa tarea (comandos más abajo).
3. **Pegar el prompt** en Antigravity. Si es una sesión nueva, pega antes el Prompt 0.
4. **Leer los cambios** que propone el agente y preguntar lo que no entiendas.
5. **Probar** en el emulador, el navegador o phpMyAdmin, según corresponda.

6 **Revisar con git status** que no se vayan a subir archivos de contraseñas (config.php) ni carpetas pesadas (build).

7 **Hacer commit y push** de la rama, con un mensaje en español que diga qué hiciste.

8 **Abrir un Pull Request** en GitHub, unirlo a main y mover la tarjeta de Trello a «Hecho».

```
git checkout main
git pull
git checkout -b feat/parte2-login
(trabajar y probar)
git status
git add .
git commit -m "feat: pantalla de login y roles"
git push -u origin feat/parte2-login
```

Prefijos de commit sugeridos: *feat:* (algo nuevo), *fix:* (corrección), *docs:* (documentación), *chore:* (configuración).

4. Calendario sugerido

Semana	Meta	Referencia en la Gantt
1: 2 a 8 nov	Preparar el entorno y completar la Parte 1	Arquitectura y flujos: 3 a 9 nov
2: 9 a 15 nov	Parte 2 (login y roles) y comenzar la Parte 3 (productos, ingreso manual)	Infraestructura backend: 10 a 14 nov
3: 16 a 22 nov	Terminar la Parte 3 manual y hacer la Parte 4 (venta rápida y resumen de ventas)	Funcionalidades principales: 13 a 27 nov
4: 23 a 29 nov	Parte 5 (alertas) y, si hay tiempo, el escáner. Parten las pruebas internas el 26	Alertas: 19 a 27 nov. Pruebas: 26 nov a 4 dic
5: 30 nov a 6 dic	Corregir fallas de las pruebas y hacer la Parte 6 (Hostinger, APK y landing)	Corrección de defectos: 3 a 9 dic
6: 7 a 13 dic	Beta con comerciantes y medición de los indicadores	Beta: 8 a 16 dic

Si una semana se atrasa: se mueve primero el escáner, luego el resumen de ventas. Nunca se sacrifica lo básico de RF001 a RF004 (productos, venta rápida, alertas y roles).

5. Las seis partes: qué hacer y cuándo está listo

Parte 1: de la base de datos a la pantalla (RNF002 y RNF003)

Qué se hace: ajustar el esquema, crear la base de prueba, una API PHP con dos rutas y una pantalla Android que lista productos. Prompts 1A a 1D del informe de decisiones (sección 10).

- [ ] En phpMyAdmin existe radar_stock_db con las tablas y los 10 productos de prueba.
- [ ] La ruta salud.php muestra el estado «ok» en el navegador.
- [ ] La ruta productos.php muestra los 10 productos en formato JSON.
- [ ] La app en el emulador muestra la lista; si apagas Apache, muestra un mensaje de error claro.
- [ ] config.php no está en Git y el commit está hecho.

Parte 2: login y roles (RF004 y RNF004)

Qué se hace: usuario administrador creado con contraseña cifrada, pantalla de login, y menú distinto según el rol.

- [ ] Las contraseñas en la base están cifradas (nunca en texto legible).
- [ ] Un login correcto entra a la app y uno incorrecto muestra un mensaje claro.
- [ ] El Administrador ve todas las secciones; el Vendedor/Colaborador solo Venta, Productos (lectura) y Alertas.
- [ ] **Importante:** si un vendedor intenta una acción de administrador, el servidor la rechaza, aunque la pantalla no la muestre.

Parte 3: productos (RF001)

Qué se hace: lista con filtro por categoría, formulario para crear y editar, eliminar solo productos sin ventas, acción «Agregar stock». Al final, el escáner para agregar stock, producto nuevo y conteo (solo administrador).

- [ ] El administrador crea, edita y elimina productos con validaciones (precio y stock válidos, código único).
- [ ] El vendedor solo puede ver y filtrar.
- [ ] Con 5.000 productos de prueba, la lista carga en menos de 2 segundos.
- [ ] «Agregar stock» suma las unidades y muestra el stock nuevo antes de confirmar.
- [ ] Extra: el escáner lee un código real y abre el flujo correcto (producto existente o producto nuevo).

Parte 4: venta rápida y resumen de ventas (RF002 y RNF001)

Qué se hace: pantalla de botones ordenados por más vendidos, filtro por categoría, resumen con total y «Confirmar venta» que descuenta el stock en una sola operación. Luego el resumen de ventas del administrador.

- [ ] Vender 3 productos descuenta exactamente esas unidades en la base y queda registrada la venta con su detalle.
- [ ] Un producto sin stock aparece en gris y no se puede agregar; si falta stock al confirmar, no se guarda nada.
- [ ] Registrar una venta toma pocos toques; cronometra el tiempo (sirve para el indicador del proyecto).
- [ ] El resumen muestra Hoy, 7 días, Este mes o un rango, con total, número de ventas y más vendidos.

Parte 5: alertas (RF003)

Qué se hace: cálculo de niveles (crítico, bajo, por vencer, vencido), pantalla de Alertas y notificación en el teléfono.

- [ ] Los productos de prueba generan cada tipo de alerta, con los mensajes definidos en el informe.
- [ ] Crítico: stock 0 o menor o igual al 25 % del stock deseado. Bajo: menor o igual al stock mínimo. Por vencer: 15 días o menos.
- [ ] Una venta que baja un producto del umbral dispara la notificación del teléfono.
- [ ] El vendedor ve las alertas pero no puede modificar umbrales.

Parte 6: Hostinger, APK y landing (RF005 y RF006)

Qué se hace: llevar la API y la base a Hostinger, compilar el APK firmado, subirlo al servidor y publicar la landing con el botón de descarga.

- [ ] En Hostinger: base creada desde el panel, script importado por phpMyAdmin, backend subido, config con los datos del hosting y HTTPS activo.
- [ ] La app apunta al dominio real (ya no a 10.0.2.2) y responde en menos de 2 segundos en una red real.
- [ ] APK firmado generado en modo release. **Guarda el archivo de firma (keystore) en un lugar seguro y nunca lo subas a GitHub.**
- [ ] El APK está en la carpeta de descargas del servidor y la landing (HTML, CSS y JS, responsive) tiene el botón visible.
- [ ] La landing explica cómo permitir «instalar apps desconocidas» en Android.
- [ ] Una persona ajena al proyecto instala el APK desde la landing, inicia sesión y registra una venta.

6. Si algo falla o te bloqueas

1. **Copia el error completo** (texto o captura) y anota qué estabas haciendo.
2. **Pídele al agente que primero explique la causa** en lenguaje simple, antes de que corrija nada.
3. **Máximo dos intentos** de arreglo seguidos. Si no funciona, vuelve al último punto bueno.
4. **Pide ayuda** con el error y el contexto (qué parte, qué prompt y qué esperabas ver).

Comandos para volver a un punto seguro:

```
git status (ver qué cambió)
git diff (ver las líneas modificadas)
git restore . (descartar los cambios sin guardar)
git log --oneline (ver los commits anteriores)
```

Atención: *git restore .* borra los cambios que no hayas guardado con commit. Úsalo solo cuando estés seguro.

7. Plantilla para cualquier prompt nuevo

Cuando una parte necesite un prompt que no esté en el informe, escríbelo con esta estructura corta:

```
OBJETIVO: lo que quiero lograr, en una frase.
YA EXISTE: lo que ya está construido y funcionando (carpetas, pantallas, rutas).
HAZ SOLO ESTO: la tarea concreta, en pasos pequeños.
REGLAS: usar solo el stack acordado; no instalar nada sin avisar; explicar cada comando.
CÓMO PROBARLO: qué debo ver cuando funcione, y espera mi confirmación.
```

8. Lo que NO se construye

Fuera de alcance	Motivo
Facturación electrónica y pagos en línea	Límite declarado en el documento del proyecto.
Varios comercios en una misma base (multiempresa)	No está en los requerimientos; complicaría el esquema.
Historial de movimientos y módulo de auditoría	No está en los requerimientos oficiales.
Publicación en Play Store	El RF006 pide descarga directa del APK desde la landing.
Notificaciones remotas desde un servidor	Basta con la notificación local del teléfono para el RF003.
Escáner en la venta	Decisión del equipo: la venta se registra solo con la pantalla rápida.

9. Evidencias que debes ir guardando para el informe

- [ ] Una captura de pantalla de cada parte terminada (app, phpMyAdmin y la ruta de la API funcionando).
- [ ] Los commits y Pull Requests de GitHub, que muestran el avance real y quién hizo qué.
- [ ] Los tiempos medidos: registrar una venta en cuaderno vs. en la app, y recuento de stock manual vs. con la app.
- [ ] El conteo de quiebres de stock antes y después de usar la app, para el indicador de reducción de quiebres.
- [ ] Una nota breve de cada decisión importante que tomaste y por qué (sirve para el registro de control de cambios).